

PROGETTO LABORATORIO DI RETI CORSO A

PIER GIORGIO CAMPANI

1 Scelte progettuali

1.1 Protocollo comunicazione TCP

La comunicazione si basa oggetti JSON, serializzati e terminati da carattere '\n'. Questa scelta consente di delimitare correttamente i messaggi su TCP, che essendo stream-oriented non preserva i confini dei messaggi. Il campo operation identifica il tipo di richiesta; il server esegue uno switch su questo campo in ClientHandler per gestire le varie richieste. Lato client si usa NIO con SocketChannel in modalità bloccante.

1.2 Gestione delle partite

Le partite sono globali: tutti i giocatori loggati partecipano alla stessa partita attiva. Uno ScheduledExecutorService avvia una nuova partita ogni gameDuration (secondi). Alla fine di ogni partita, i giocatori loggati vengono automaticamente iscritti alla nuova partita tramite addLoggedUsersToGame. Per il singolo giocatore la partita termina in caso di vittoria (3 proposte corrette), in caso di sconfitta (4 proposte errate) o nel caso in cui il tempo scade. Le proposte malformate (ad esempio: parole non presenti nella lista della partita corrente) non vengono contate come errore nello stato della partita.

1.3 Punteggi

Il punteggio di partita (PlayerGameState.score) viene aggiornato in tempo reale. Il punteggio massimo è +18. Il punteggio globale (User.totalScore) viene aggiornato solo al termine della partita tramite finalizeWin o finalizeLoss per vittoria o sconfitta e tramite finalizeGame per i giocatori che non hanno terminato entro il tempo.

1.4 Persistenza

I dati sono persistiti in due file JSON con pretty-printing: users.json che mantiene gli utenti, le loro statistiche e il loro stato nella partita in corso; game_state.json contiene NextGameIndex (permette di ripartire da dove eravamo arrivati in caso di interruzione del server) e le statistiche di tutte le partite. La persistenza avviene periodicamente tramite PersistenceTask.

1.5 Notifiche UDP asincrone

Le notifiche asincrone inviate vengono inviate via UDP. Il client comunica la propria porta UDP effimera al server nel messaggio di login – evitando conflitti di porta tra più client. Il server usa socket effimere per ogni invio in UDPNotifier. Il client avvia un thread daemon (UDPListener) dopo il login. Il server può inviare due tipi di notifiche: GameEndNotification (esito personale della partita, statistiche, classifica della partita) e NewGameNotification (informazioni relative alla nuova partita), distinte dal campo type nel JSON.

1.6 Caricamento partite

I file JSON delle partite vengono lette tramite JsonReader. Al riavvio le partite già giocate vengono saltate con skipValue().

2 Schema dei Thread

2.1 Lato server

- Main: Accetta connessioni TCP.

- **ClientHandler**: Uno per ogni client che si connette. Si occupa di gestire la comunicazione TCP con il relativo client.
- **GameScheduler**: Ha il compito di avviare una partita (`startNewGame`) al termine di ogni partita (ogni `gameDuration`)
- **PersistenceTask**: Salva gli utenti e lo stato ogni `persistenceInterval`

2.2 Lato client

- **Main**: Gestisce i comandi del client e la comunicazione con il server.
- **UDPListener**: Riceve le notifiche UDP asincrone dal server

3 STRUTTURE DATI

3.1 Lato Server

- `users` è una `ConcurrentHashMap <String, User>`. Ovvero mappa username in User. Permette accesso concorrente da più ClientHandler.
- `udpClients` è una `ConcurrentHashMap <String, SocketAddress>`. Mappa username in un indirizzo UDP per le notifiche asincrone.
- `gameStats` è una `ConcurrentHashMap<Integer, GameStats>` che mantiene le statistiche di ogni partita. Viene caricata e salvata in JSON.
- `pastGames` è una `HashMap<Integer, PlayerGameState>`. Mantiene le statistiche personali dei giocatori nelle partite già giocate. E' protetta da `synchronized`.
- `correctGroups` è un `ArrayList<Group>` contenente i gruppi trovati dal giocatore nella partita corrente.
- `shuffledWords` è un `ArrayList<String>` contiene le 16 parole mescolate della partita corrente.
- `mistakeHistogram (int[6])` dove gli indici 0-4 corrispondono agli errori per partita vinta o persa, l'indice 5 per le partite non terminate in tempo.

Usiamo `ConcurrentHashMap` per `users`, `udpClients` e `gameStats` poichè sono strutture condivise tra più thread contemporaneamente. `ConcurrentHashMap` garantisce thread-safety tramite fine grain locking.

Usiamo `HashMap` per `pastGames` in `User` poiché appartiene a un singolo utente e l'accesso è sempre protetto da `synchronized`.

`ArrayList` per `correctGroups` e `shuffledWords`: accesso sequenziale, ordine rilevante e sono protette da `synchronized` in `GameManager`.

3.2 Lato Client

Manteniamo i buffer NIO, allocati una volta sola e riutilizzati per tutta la sessione, vengono usati da un solo thread. Il `DatagramSocket UDP` viene creato al login e chiuso al logout o exit.

4. Primitive di sincronizzazione

4.1 UserManager

Usiamo `putIfAbsent` della `concurrentHashMap` per evitare registrazione duplicate.

Usiamo `synchronized(user)` in login e logout bloccando il singolo utente.

Per `updateCredential` usiamo `synchronized(this)` per rendere atomica la sequenza di operazioni sulla mappa `containsKey`, `remove`, `put`.

Negli altri metodi in cui dobbiamo effettuare operazioni su più utenti o scrittura su file usiamo `synchronized(this)`.

4.2 GameManager

Tutti i metodi pubblici sono synchronized (coarse grain). Questa scelta garantisce consistenza di currentGame e shuffledWords, modificati dallo scheduler e letti dai ClientHandler senza rischi di race condition. I metodi di GameStats anch'essi synchronized poiché condivisi tra GameManager e i thread ClientHandler.

4.3 Scelte tra fine grain e coarse grain

In UserManager si è adottato un approccio misto con fine grain per operazioni frequenti come login e logout che coinvolgono il singolo utente e coarse grain per operazioni composite come in updateCredentials che opera su più utenti o sulla mappa.

5 Istruzioni per la compilazione

5.1 Librerie esterne

E' stato utilizzato Gson 2.13.2 (gson-2.13.2.jar) per la serializzazione/deserializzazione JSON. Il file JAR si trova nella cartella principale del progetto

5.2 Struttura del progetto

- server/ - codice sorgente server
- server/game – classi di gestione partite
- client/ - codice sorgente client
- messages/ - classi messaggi condivise
- config/ - configurazione server e configurazione client
- data/ - file JSON
- gson.2.13.2.jar
- ServerMain.jar
- ClientMain.jar

5.3 Compilazione

Server: javac -cp "gson-2.13.2.jar" -d out server/game/* server/*.java messages/*.java messages/responses/*.java messages/requests/*.java

Client: javac -cp "gson-2.13.2.jar" -d out client/*.java messages/*.java messages/requests/*.java messages/responses/*.java

5.4 Esecuzione

Server: java -cp "out;gson-2.13.2.jar" server.ServerMain

Client: java -cp "out;gson-2.13.2.jar" client.ClientMain

5.5 Esecuzione JAR

Server: java -jar ServerMain.jar

Client: java -jar ClientMain.jar